

TPM SPI Link Encryption 검증 절차서

1. 개요

항목	내용
목적	tpm-spi-link-encryption-summary 4장 적용 범위 표의 각 항목이 TPM SPI 버스에서 비밀을 평문으로 보내지 않는다는 것을 디바이스에서 확인
대상 이미지	tpm-spi-encryption 브랜치 빌드 (커널 CONFIG_TCG_TPM2_HMAC=y)
필요 권한	디바이스 root 셸 (시리얼 콘솔 또는 SSH)
소요 시간	전체 약 20분 (OTA 재부팅 항목이 가장 오래 걸림)

1.1 판정 원리

항목마다 아래 세 가지를 확인한다.

1. **평문 부재**: 알려진 비밀 값(봉인한 값 또는 해제된 값)의 hex가 TPM으로 오간 명령·응답 버퍼 어디에도 없어야 한다.
2. **세션 속성**: 비밀이 실리는 명령(Create, Unseal, GetRandom 등)이 세션 태그 0x8002로 나가고, 입력이 비밀이면 decrypt, 출력이 비밀이면 encrypt 속성이 켜져 있어야 한다.
3. **대조군**: 같은 방법으로 세션 없이 보낸 명령에서는 비밀이 평문으로 보여야 한다 (3장). 그래야 1번의 “없음”이 측정 실패가 아니라는 근거가 된다.

1.2 측정 위치

- **사용자 공간**: tpm2-tss의 TCTI 트레이스(TSS2_LOG=tcti+debug)가 TPM으로 보내는 명령 버퍼와 받은 응답 버퍼를 hex로 기록한다.
 - TCTI는 device:/dev/tpmrm0로 지정한다. 기본값인 tpm2-abrmd 경유 시에는 클라이언트 쪽에 버퍼가 기록되지 않는다.
 - /dev/tpmrm0의 커널 리소스 매니저는 핸들 영역과 컨텍스트만 바꾸고 파라미터 영역은 그대로 TPM SPI 드라이버로 넘긴다. 따라서 이 버퍼의 파라미터 영역은 SPI 버스에 실리는 바이트와 같다.
 - initramfs 단계(부팅 중 rootfs·FDE 키 해제)는 트레이스할 수 없으므로, 부팅 로그로 경로를 확인하고 같은 라이브러리 함수를 런타임에 다시 실행해 측정한다.
- **커널**: tracefs kprobe로 tpm_transmit()에 들어가는 명령 버퍼의 헤더와 세션 속성 바이트를 읽는다.

1.3 주의 사항

- 시험용 비밀은 모두 더미 값을 쓴다. 실제 PSK/PPK가 봉인돼 있으면 10장의 봉인 시험은 건너뛴다.
- 판정이 실패하면 트레이스 로그에 평문 비밀이 남는다. 시험이 끝나면 12장의 정리 절차로 반드시 지운다.
- 이 절차는 시험용 핸들 0x810200f0, 0x810200f1을 잠시 만들었다가 지운다. 이 두 핸들이 이미 쓰이고 있으면 다른 빈 핸들로 바꾼다.

2. 준비

부록 A의 tpm-wire.sh를 디바이스의 /run/tpm-wire.sh로 복사한 뒤, root 셸에서 아래를 실행한다. 이후 모든 절차는 이 셸에서 이어서 실행한다.

```

chmod 755 /run/tpm-wire.sh
W=/run/tpm-wire.sh
export TPM2T00LS_TCTI=device:/dev/tpmrm0 TSS2_TCTI=device:/dev/tpmrm0
. /usr/lib/shiba/tpm-algo.sh
. /usr/lib/shiba/fde-kek.sh
detect_tpm_algo
B=$(grep -o 'boot_bank=[0-9]' /proc/cmdline | cut -d= -f2)
VARIANT=$(cat /usr/lib/shiba/shiba-variant)
echo "bank=$B variant=$VARIANT alg=$TPM_HASH_ALG"
trace_on() { rm -f /run/tr-$1.log; export TSS2_LOG=tcti+debug TSS2_LOGFILE=/run/tr-$1.log; }
trace_off() { unset TSS2_LOG TSS2_LOGFILE; }
wipe() { for f in "$@"; do [ -f "$f" ] || continue; chmod 600 "$f"; dd if=/dev/zero of="$f"
        bs=$(wc -c <"$f") count=1 conv=notrunc 2>/dev/null; rm -f "$f"; done; }

```

tpm-wire.sh 사용법:

명령	출력
\$W bufs <trace>	버퍼마다 한 줄: C(명령)/R(응답), 태그, 명령 코드 또는 응답 코드, 전체 hex
\$W find <trace> <hex>	<hex>를 포함한 버퍼 수 (평문 부재면 0)
\$W sess <trace>	Create/Unseal/Load/Sign/GetRandom/CreatePrimary 명령의 세션 핸들과 속성
\$W hex <file>	파일 내용을 hex로 출력

sess 출력 읽는 법은 부록 B를 참고한다.

3. 대조 시험 (세션 없음 → 평문이 보여야 함)

```

trace_on ctl-rand
tpm2_getrandom -o /run/r.bin 8 >/dev/null
trace_off
echo "난수 평문 포함 버퍼: $($W find /run/tr-ctl-rand.log $($W hex /run/r.bin))"

printf 'ZqT7-control-0123456789abcdefghij' > /run/v.bin
trace_on ctl-create
tpm2_createprimary -C o -c /run/ctl.ctx >/dev/null
tpm2_create -C /run/ctl.ctx -i /run/v.bin -u /run/ctl.pub -r /run/ctl.priv >/dev/null
trace_off
echo "비밀 평문 포함 버퍼: $($W find /run/tr-ctl-create.log $($W hex /run/v.bin))"
$W sess /run/tr-ctl-create.log
tpm2_flushcontext -t >/dev/null; rm -f /run/ctl.* /run/v.bin /run/r.bin

```

합격 기준

- 두 find 결과가 모두 1 이상
- sess 출력의 Create에 decrypt 속성이 없음 (예: Create [02000001 continue,] — 세션은 있지만 파라미터 암호화가 꺼져 있음)

이 결과가 나오지 않으면 트레이스가 제대로 잡히지 않은 것이므로 이후 판정을 믿을 수 없다. 2장의 환경 변수를 다시 확인한다.

4. 커널: hwrng TPM2_GetRandom

```
zcat /proc/config.gz | grep CONFIG_TCG_TPM2_HMAC
cat /sys/class/tpm/tpm0/null_name
T=/sys/kernel/tracing; [ -d $T/events ] || mount -t tracefs nodev $T
echo 0 > $T/tracing_on; echo > $T/trace; echo > $T/kprobe_events
echo 'p:tpmcmd tpm_transmit tag=+0(%r1):x16 cc=+6(%r1):x32 attr=+52(%r1):x8' >> $T/
kprobe_events
echo 'p:hmac_fill tpm_buf_fill_hmac_session' >> $T/kprobe_events
echo 'p:hmac_check tpm_buf_check_hmac_response' >> $T/kprobe_events
echo 1 > $T/events/kprobes/enable; echo 1 > $T/tracing_on
head -c 64 /dev/hwrng > /dev/null
tpm2_getrandom 8 > /dev/null
echo 0 > $T/tracing_on
grep -v '^#' $T/trace | sed -E 's/^(^+)-[0-9]+.*(tpmcmd|hmac_fill|hmac_check): \
([~]*~)/\1 \2/' | sort | uniq -c
echo 0 > $T/events/kprobes/enable; echo > $T/kprobe_events
```

kprobe는 리틀 엔디언으로 읽으므로 tag=0x280은 0x8002(세션 있음), tag=0x180은 0x8001(세션 없음), cc=0x7b010000은 0x0000017B(GetRandom)이다. attr은 세션 속성 바이트이다(부록 B).

합격 기준

- CONFIG_TCG_TPM2_HMAC=y, null_name이 000b로 시작하는 값
- head 프로세스(hwrng 읽기)의 명령이 tag=0x280 cc=0x7b010000 attr=0x41 (continue + encrypt: 응답 난수 암호화)
- head의 hmac_fill과 hmac_check 횟수가 tpmcmd 횟수와 같음 (명령마다 HMAC 생성·검증)
- 대조: 사용자 공간 tpm2_getrandom(kworker로 표시)은 tag=0x180 cc=0x7b010000 attr=0x0

64B 읽기는 32B씩 2개 명령으로 나뉜다. 커널 세션은 hwrng 호출 사이에 유지되지만(lazy flush) 사용자 공간 명령이 끼어들면 닫힌다. 그래서 직전에 다른 TPM 명령을 실행했다면 head 아래에 세션 재수립 명령 3개가 함께 보인다:

cc=0x61010000 (ContextLoad, null key 로드), cc=0x76010000 (StartAuthSession, salt 전달), cc=0x65010000 (FlushContext). 이 명령들은 비밀을 신지 않으며 attr 값은 의미가 없다.

5. 커널: TPM2_PCR_Extend

현재 커널에는 TPM2_PCR_Extend를 부르는 기능(IMA, trusted keys)이 꺼져 있다. 호출이 없다는 것을 확인한다.

```
zcat /proc/config.gz | grep -E '^CONFIG_IMA=|^CONFIG_TRUSTED_KEYS=' || echo "IMA,
TRUSTED_KEYS 꺼짐"
grep -w tpm2_pcr_extend /proc/kallsyms
T=/sys/kernel/tracing
echo 'p:pcr tpm2_pcr_extend' > $T/kprobe_events
echo 1 > $T/events/kprobes/enable; echo > $T/trace; echo 1 > $T/tracing_on
sleep 60
echo 0 > $T/tracing_on
echo "tpm2_pcr_extend 호출 수: $(grep -c ' pcr:' $T/trace)"
echo 0 > $T/events/kprobes/enable; echo > $T/kprobe_events
```

합격 기준: IMA·TRUSTED_KEYS가 꺼져 있고 호출 수 0. 이 함수가 불리면 HMAC 세션으로 보내도록 패치돼 있으나(0005-tpm-hmac-sessions.patch의 tpm2_pcr_extend), 현재 호출 경로가 없으므로 동적 측정 대상이 아니다.

6. 사용자 공간: rootfs LUKS 키 해제 (init-dmccrypt.sh)

부팅 중 initramfs가 실행하므로, 부팅 로그로 경로를 확인한 뒤 같은 정책(PolicyNV + PolicyPCR)과 같은 세션 함수 (tpm_start_encrypted_policy_session)를 쓰는 _fde_unseal_sealed_blob_once로 현재 bank의 rootfs 키를 다시 해제해 측정한다.

```
journalctl -b | grep 'dm-crypt:.*opening encrypted policy session'
trace_on rootfs
_fde_unseal_sealed_blob_once $(printf '0x%x' $((0x81020000 + B))) /run/k.bin 64; echo "rc=$?"
trace_off
echo "평문 포함 버퍼: $($W find /run/tr-rootfs.log $($W hex /run/k.bin))"
$W sess /run/tr-rootfs.log
wipe /run/k.bin
```

합격 기준

- 부팅 로그에 opening encrypted policy session (sha384, EK-salted) 1줄
- rc=0, 평문 포함 버퍼 0
- Unseal [03xxxxxx continue,encrypt,] (정책 세션, 응답 암호화)

7. 사용자 공간: overlay-data FDE·FE 키 봉인/해제 (fde-kek-lib.sh, init-fde.sh, fe-lib.sh)

7.1 해제

grey는 overlay KEK, red는 공유 게이트(overlay-data)와 FE day-MAC 키를 해제한다.

```
if [ "$VARIANT" = grey ]; then
    trace_on fde-unseal
    fde_grey_unseal_kek $(printf '0x%x' $((0x81020022 + B))) /run/k.bin; echo "rc=$?"
    trace_off
    echo "평문 포함 버퍼: $($W find /run/tr-fde-unseal.log $($W hex /run/k.bin))"
    wipe /run/k.bin
else
    trace_on fde-unseal
    fde_unseal_to_salt_and_submask_b $(printf '0x%x' $((0x81020020 + B))) /run/salt.bin /run/
    sb.bin; echo "rc=$?"
    trace_off
    echo "평문 포함 버퍼: salt=$($W find /run/tr-fde-unseal.log $($W hex /run/salt.bin))
    submask_B=$($W find /run/tr-fde-unseal.log $($W hex /run/sb.bin))"
    wipe /run/salt.bin /run/sb.bin
fi
$W sess /run/tr-fde-unseal.log
```

red에서 FE가 프로비저닝돼 있으면, 위 else 블록의 핸들을 0x81020030 + B (FE day-MAC 키)로 바꿔 한 번 더 실행한다.

7.2 봉인

FDE 프로비저닝도 initramfs에서 실행되므로, 같은 봉인 함수 _fde_seal_blob으로 더미 값을 시험용 핸들에 봉인해 측정한다.

```
head -c 96 /dev/urandom > /run/sb.test
trace_on fde-seal
_fde_seal_blob /run/sb.test 0x810200f1; echo "rc=$?"
trace_off
echo "평문 포함 버퍼: $($W find /run/tr-fde-seal.log $($W hex /run/sb.test))"
```

```
$W sess /run/tr-fde-seal.log
tpm2_evictcontrol -C o -c 0x810200f1 >/dev/null && echo "시험 핸들 삭제"
wipe /run/sb.test
```

합격 기준

- 해제: rc=0, 평문 포함 버퍼 0, Unseal [03xxxxxx continue,encrypt,]
- 봉인: rc=0, 평문 포함 버퍼 0, Create [02xxxxxx continue,decrypt,encrypt,audit,] (HMAC 세션, 입력 암호화)

8. 사용자 공간: OTA 재봉인 (pcr-predict-reseal.sh)

OTA가 실제로 부르는 스크립트로 더미 값을 시험용 핸들에 봉인한다. `--verify-against-live`는 현재 bank를 대상으로 예측 PCR이 실제 PCR과 같은지 확인하는 옵션이라, 펌웨어를 바꾸지 않고 시험할 수 있다.

```
printf 'ZqT7-ota-test-0123456789abcdefghij' > /run/ota.bin
HV=$(($W hex /run/ota.bin)
trace_on ota
pcr-predict-reseal.sh $B --seal 0x810200f0:/run/ota.bin --verify-against-live > /run/ota.out
2>&1; echo "rc=$?"
trace_off
tail -1 /run/ota.out
echo "평문 포함 버퍼: $($W find /run/tr-ota.log $HV)"
$W sess /run/tr-ota.log
tpm2_evictcontrol -C o -c 0x810200f0 >/dev/null && echo "시험 핸들 삭제"
rm -f /run/ota.bin /run/ota.out
```

스크립트가 봉인 후 입력 파일을 지우므로 hex(HV)를 먼저 계산해 둔다.

합격 기준: rc=0, SUCCESS: secret sealed to 0x810200f0, 평문 포함 버퍼 0, Create [02xxxxxx continue,decrypt,encrypt,audit,]

재봉인 모드의 해제 쪽(`try_unseal_compound`)은 6장과 같은 세션 함수를 쓴다.

9. 사용자 공간: VPN PKCS#11 PIN (vpn-pkcs11-pin.sh)

부팅 때 실행되는 서비스 스크립트를 다시 실행한다. 봉인된 PIN(8바이트)을 해제해 /run/vpn/pkcs11-pin.tmp에 hex로 쓰므로, 그 hex를 응답 버퍼에서 찾는다.

```
trace_on pin
/usr/sbin/vpn-pkcs11-pin.sh >/dev/null 2>&1; echo "rc=$?"
trace_off
P=$(tr 'A-F' 'a-f' < /run/vpn/pkcs11-pin.tmp | tr -d '\n')
echo "PIN 길이=${#P} 평문 포함 버퍼: $($W find /run/tr-pin.log $P)"
$W sess /run/tr-pin.log
```

합격 기준: rc=0, PIN 길이 16, 평문 포함 버퍼 0, Unseal [03xxxxxx continue,encrypt,]

PIN을 처음 만드는 경로(봉인)는 첫 부팅에만 실행되며, 7.2-8장과 같은 `tpm_start_encrypted_hmac_session`을 쓴다.

10. 사용자 공간: VPN PSK/PPK (tpm-seal-secret.sh, tpm-unseal-secret.sh)

실제 PSK/PPK가 봉인돼 있으면 봉인 시험이 그 값을 덮어쓰므로 건너뛴다.

```

for t in psk ppk; do
    h=0x81010101; [ $t = ppk ] && h=0x81010201
    if tpm2_readpublic -c $h >/dev/null 2>&1; then echo "[ $t ] 이미 프로비저닝됨 - 건너뛴";
        continue; fi
    V="ZqT7-$t-test-0123456789abcdefghijklmnopqrstuvwxyz"; printf '%s' "$V" > /run/v.bin
    trace_on $t-seal
    printf '%s' "$V" | tpm-seal-secret.sh $t - >/dev/null 2>&1; echo "[ $t ] seal rc=$?"
    trace_off
    trace_on $t-unseal
    R=$(tpm-unseal-secret.sh $t 2>/dev/null)
    trace_off
    [ "$R" = "$V" ] && echo "[ $t ] 봉인/해제 값 일치"
    echo "[ $t ] 평문 포함 버퍼: 봉인=$(($W find /run/tr-$t-seal.log $(($W hex /run/v.bin)) 해제=$(($W
        find /run/tr-$t-unseal.log $(($W hex /run/v.bin)))"
    $W sess /run/tr-$t-seal.log; $W sess /run/tr-$t-unseal.log
    tpm-seal-secret.sh $t "" >/dev/null 2>&1 && echo "[ $t ] 시험 값 삭제"
    rm -f /run/v.bin
done

```

합격 기준 (psk, ppk 각각)

- 봉인 rc=0, 값 일치, 평문 포함 버퍼 봉인 0·해제 0
- Create [02xxxxxx continue,decrypt,encrypt,audit,], Unseal [03xxxxxx continue,encrypt,]

SCLI가 비밀을 stdin으로 넘기는지(프로세스 목록 노출 없음)는 이 문서의 범위(SPI 버스) 밖이다.

11. PKCS#11: 토큰/키 프로비저닝과 런타임 서명 (libtpm2_pkcs11)

빈 슬롯에 시험용 토큰을 만들어 프로비저닝(pkcs11-functions.inc와 같은 pkcs11-tool 명령)과 서명을 측정한다. 토큰 DB를 백업했다가 시험 후 그대로 되돌린다. libtpm2_pkcs11은 TCTI를 TPM2_PKCS11_TCTI로 받는다.

```

export TPM2_PKCS11_TCTI=device:/dev/tpmrm0 TPM2_PKCS11_STORE=/var/lib/tpm2-pkcs11
    TPM2_PKCS11_LOG_LEVEL=0
M=/usr/lib/pkcs11/libtpm2_pkcs11.so; DB=$TPM2_PKCS11_STORE/tpm2_pkcs11.sqlite3
cp -p $DB /run/p11db.bak
S0='ZqT7wireS0pin01'; UP='ZqT7wireUSERpin2'
printf '%s' "$S0" > /run/so.bin; printf '%s' "$UP" > /run/up.bin
SLOT=$(pkcs11-tool --module $M -L 2>/dev/null | awk '/^Slot [0-9]+ \(\0x/{s=$3; gsub(/
    [()]/, "", s)} /token state:[[:space:]]*uninitialized/{print s; exit}')
echo "빈 슬롯=$SLOT"
trace_on p11prov
pkcs11-tool --module $M --slot $SLOT --init-token --label wiretest --so-pin "$S0" >/dev/null
    2>&1; echo "init-token rc=$?"
pkcs11-tool --module $M --token-label wiretest --login --login-type so --so-pin "$S0" --init-
    pin --pin "$UP" >/dev/null 2>&1; echo "init-pin rc=$?"
pkcs11-tool --module $M --token-label wiretest --login --pin "$UP" --keypairgen --key-type
    EC:secp384r1 --label wk --id 01 >/dev/null 2>&1; echo "keypairgen rc=$?"
trace_off
echo "프로비저닝 평문 포함 버퍼: S0 PIN=$(($W find /run/tr-p11prov.log $(($W hex /run/so.bin)) user
    PIN=$(($W find /run/tr-p11prov.log $(($W hex /run/up.bin)))"
$W sess /run/tr-p11prov.log | sort | uniq -c
printf 'hello' > /run/msg.bin
trace_on p11sign
pkcs11-tool --module $M --token-label wiretest --login --pin "$UP" --sign --mechanism ECDSA-
    SHA384 --id 01 -i /run/msg.bin -o /run/sig.bin >/dev/null 2>&1; echo "sign rc=$?"
trace_off
echo "서명 평문 포함 버퍼: user PIN=$(($W find /run/tr-p11sign.log $(($W hex /run/up.bin)))"
$W sess /run/tr-p11sign.log | sort | uniq -c

```

```
cp -p /run/p11db.bak $DB && echo "토큰 DB 복구"
rm -f /run/p11db.bak /run/so.bin /run/up.bin /run/msg.bin /run/sig.bin
```

합격 기준

- 각 단계 rc=0, 평문 포함 버퍼 모두 0
- 프로비저닝: Create·Load가 [02xxxxxx continue,decrypt,encrypt,], Unseal이 [02xxxxxx continue,encrypt,]
- 서명: Load가 [02xxxxxx continue,decrypt,encrypt,], Unseal이 [02xxxxxx continue,encrypt,], Sign이 [02xxxxxx continue,decrypt,]
- 복구 후 pkcs11-tool --module \$M -L에 wiretest 토큰이 없음

12. 정리

```
unset TSS2_LOG TSS2_LOGFILE TPM2T00LS_TCTI TSS2_TCTI TPM2_PKCS11_TCTI
for f in /run/tr-*.log; do wipe "$f"; done
rm -f /run/tpm-wire.sh
for h in 0x810200f0 0x810200f1; do tpm2_readpublic -c $h >/dev/null 2>&1 && echo "남은 시험 행  
들: $h"; done
```

13. 결과 기록

장	항목	기준	실측 (2026-10-07, grey shiba-25q3m1, 이미지 20261007015314)
3	대조군	평문 보임	GetRandom 난수 1, Create 비밀 1
4	커널 hwrng	tag=0x280 attr=0x41, HMAC 생성 ·검증	일치 (대조 tag=0x180 attr=0x0)
5	커널 PCR_Extend	호출 0	0
6	rootfs 키 해제	평문 0, Unseal encrypt	0, [03000001 continue,encrypt,]
7.1	FDE 해제	평문 0, Unseal encrypt	grey KEK: 0, [03000001 continue,encrypt,]
7.2	FDE 봉인	평문 0, Create decrypt	0, [02000001 continue,decrypt,encrypt,audit,]
8	OTA 재봉인	평문 0, Create decrypt	0, [02000001 continue,decrypt,encrypt,audit,]
9	VPN PIN	평문 0, Unseal encrypt	0, [03000001 continue,encrypt,]
10	PSK/PPK	평문 0, Create decrypt / Unseal encrypt	0 / 0, 속성 일치
11	PKCS#11	PIN 평문 0, 세션 속성	SO PIN 0, user PIN 0, 속성 일치

red 전용 항목(공유 게이트, FE day-MAC)은 같은 함수(fde_unseal_to_salt_and_submask_b)를 쓰며, red 보드에서 7.1의 else 블록으로 측정한다.

부록 A. tpm-wire.sh

```
#!/bin/sh
# tpm-wire.sh - split a TSS2 tcti trace into TPM wire buffers
# tpm-wire.sh bufs <trace>          C|R <tag> <cc|rc> <hex>, one per buffer
# tpm-wire.sh find <trace> <hex>    number of buffers containing <hex>
# tpm-wire.sh sess <trace>          session attributes of Create/Unseal/Load/Sign/...
# tpm-wire.sh hex <file>            file contents as hex

bufs() {
    awk '
    function emit() { if (k != "" && b != "") print k, substr(b, 1, 4), substr(b, 13, 8), b; k
        = ""; b = "" }
    /command buffer: \(size=/ { emit(); k = "C"; next }
    /Response Received \(size=/ { emit(); k = "R"; next }
    k != "" && /^[0-9a-f][0-9a-f][0-9a-f][0-9a-f]: / { h = substr($0, 7, 32); gsub(/ /, "", h);
        b = b h; next }
    k != "" { emit() }
    END { emit() }' "$1"
}

sess() {
    bufs "$1" | awk '
    function hx(s, i, v) { v = 0; for (i = 1; i <= length(s); i++) v = v * 16 +
        index("0123456789abcdef", substr(s, i, 1)) - 1; return v }
    function attr(a, s) { s = ""; if (a % 2) s = s "continue,"; if (int(a / 32) % 2) s = s
        "decrypt,"; if (int(a / 64) % 2) s = s "encrypt,"; if (int(a / 128) % 2) s = s
        "audit,"; return s }
    BEGIN {
        nh["00000153"] = 1; nm["00000153"] = "Create"; nh["0000015e"] = 1; nm["0000015e"] =
            "Unseal"
        nh["0000017b"] = 0; nm["0000017b"] = "GetRandom"; nh["00000157"] = 1; nm["00000157"] =
            "Load"
        nh["0000015d"] = 1; nm["0000015d"] = "Sign"; nh["00000131"] = 1; nm["00000131"] =
            "CreatePrimary"
    }
    $1 == "C" && $2 == "8002" && ($3 in nh) {
        b = $4; p = 21 + nh[$3] * 8; end = p + 8 + hx(substr(b, p, 8)) * 2; p += 8; out = ""
        while (p < end) {
            h = substr(b, p, 8); p += 8; n = hx(substr(b, p, 4)); p += 4 + n * 2
            a = hx(substr(b, p, 2)); p += 2; m = hx(substr(b, p, 4)); p += 4 + m * 2
            out = out " [" h " " attr(a) "]"
        }
        print nm[$3] out
    }'
}

case "$1" in
    bufs) bufs "$2" ;;
    find) [ -n "$3" ] || { echo "find: empty hex" >&2; exit 2; }
        bufs "$2" | awk -v h="$3" 'index($4, h) { n++ } END { print n + 0 }' ;;
    sess) sess "$2" ;;
    hex) od -An -v -tx1 "$2" | tr -d ' \n' ;;
    *) echo "usage: $0 bufs|find|sess|hex ..." >&2; exit 2 ;;
esac
```

부록 B. TPM 값 읽는 법

값	의미
태그 8001	세션 없음

값	의미
태그 8002	세션 있음
세션 핸들 02xxxxxx	HMAC 세션
세션 핸들 03xxxxxx	정책(policy) 세션
세션 핸들 40000009	패스워드 세션 (암호화 불가, 인증값이 평문)
속성 continue (0x01)	명령 후에도 세션 유지
속성 decrypt (0x20)	명령의 첫 파라미터를 암호화해 보냄 (TPM이 복호화)
속성 encrypt (0x40)	TPM이 응답의 첫 파라미터를 암호화해 보냄
속성 audit (0x80)	감사 세션 (tpm2-tools --audit-session이 함께 켜)
명령 코드	0x131 CreatePrimary, 0x153 Create, 0x157 Load, 0x15D Sign, 0x15E Unseal, 0x17B GetRandom

세션이 EK(또는 owner) primary로 salt돼 있어야 세션 키가 버스에 드러나지 않는다. 사용자 공간 세션은 tpm-algo.sh의 tpm_start_encrypted_*_session이 /run/tpm-salt.ctx 키로, 커널 세션은 null hierarchy primary로 salt한다.